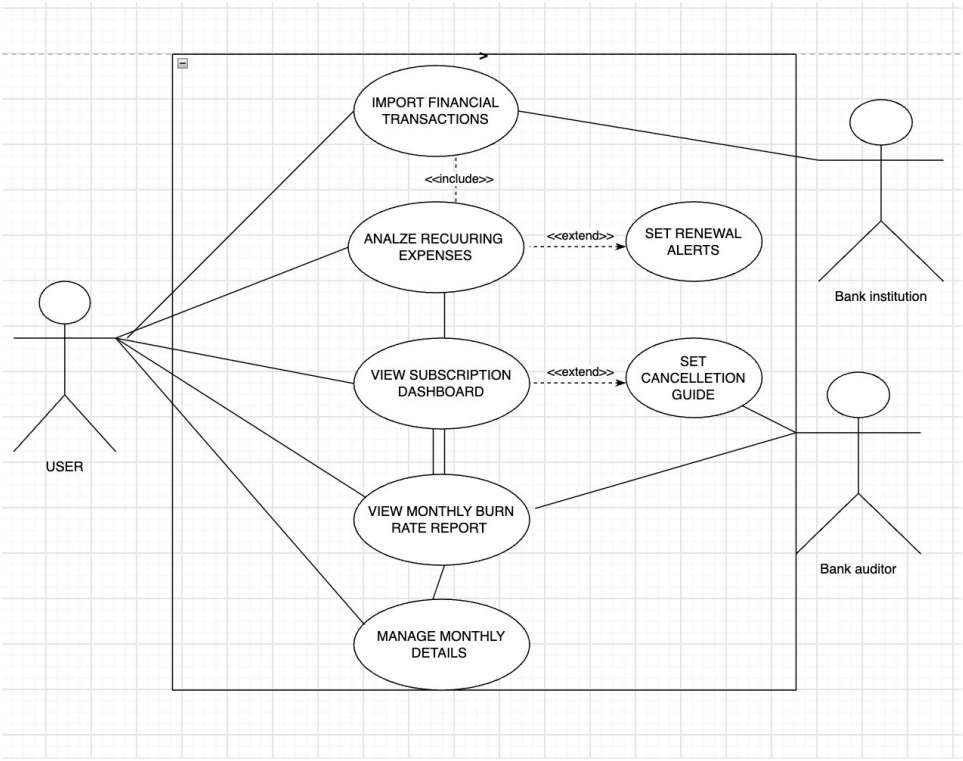
SE Lab 1: Personal Subscription & Recurring Expense Auditor

Requirements Engineering & UML Use-Case Modelling

1. Requirements Table

ID	Type	Description	Priority	Acceptance Criteria	Rationale
FR-001	Functional	Import transaction data from CSV/OFX files, extract date, amount and merchant fields, and store each record with a unique ID without exposing account credentials.	High	All fields extracted correctly; no data loss; no plaintext credentials stored.	Secure, accurate import is the basis for all later analysis.
FR-002	Functional	Analyze transaction history to detect recurring payment patterns, classify them by frequency (weekly, monthly, annual), and estimate renewal dates.	High	Detection accuracy above 85%; renewal dates recorded; confidence score assigned to each match.	Automated detection removes the need to track subscriptions manually.
FR-003	Functional	Notify the user seven days before a subscription renews, including the name, amount and a cancellation link.	High	Alerts sent on schedule with complete details, delivered via in-app and email.	Timely alerts let users cancel unwanted subscriptions before being charged.
FR-004	Functional	Compute total monthly subscription spend and break it down by category (e.g., streaming, finance, productivity).	Medium	Calculations accurate within ±2%; category totals complete; report updates on new imports.	Spend visibility helps users manage recurring expenses.
FR-005	Functional	Provide step-by-step cancellation instructions and direct links for detected subscriptions.	Medium	Guides available for at least 90% of detected subscriptions; links and steps remain accurate.	Simplifies the cancellation process for the user.
NFR-001	Non-Functional	Encrypt sensitive data at rest (AES-256) and in transit (TLS 1.2+); never store banking credentials in plaintext.	Critical	Security audit confirms no plaintext credentials and verified encryption standards.	Protects sensitive financial data from unauthorized access.
NFR-002	Non-Functional	Complete file imports within 2 seconds, pattern detection within 5 seconds for five years of history, and API responses within 1000 ms at P95 under 100 concurrent users.	High	Benchmark tests confirm all thresholds are met with no timeouts under load.	Keeps the application responsive as data volume grows.

2. UML Use-Case Diagram



Actors: Individual User (primary), Finance Auditor (secondary).

Use cases: Import Data, Analyze Recurring, View Dashboard, View Burn Rate, Get Renewal Alert, Cancellation Guide.

Relationships: «include» – Analyze Recurring includes Import Data. «extend» – View Burn Rate extends Analyze Recurring.

3. Main Success Scenario

Use Case Name: Analyze Recurring Expenses

Primary Actor: Individual User

Secondary Actor: Finance Auditor (audit logs)

Preconditions

User is logged in with at least 3 months of imported transaction history; each transaction has a date, amount and merchant name.

Main Success Scenario

1. User clicks “Analyze Subscriptions.”
2. System loads the last 3+ months of imported transactions.
3. System groups the transactions by merchant and checks the gap between payments.
4. System looks for repeating charges: similar amount (within ±5%) and a steady gap (variation under 5 days).
5. System labels each match as weekly, bi-weekly, monthly, quarterly or annual.

6. System predicts the next renewal date from the last payment date and the detected gap.
7. System scores how confident it is, based on how often and how regularly the charge repeats.
8. System keeps only matches with 70% confidence or higher.
9. System sorts the results by category and totals the monthly spend.
10. System shows the user a dashboard with the subscription list, total spend and category breakdown.
11. User checks the results and flags any incorrect matches.
12. System updates the dashboard, removing the flagged items.

Postconditions

Recurring charges are stored with their dates, amounts and confidence scores; the monthly burn rate is available for reporting; data is ready for renewal alerts (FR-003).

Alternate Flow: Insufficient History

Trigger: Less than 3 months of imported data.

1. System tells the user more data is needed and shows how much history is currently available.
2. User chooses to continue with lower confidence (50% or higher) or to import more data.
3. If the user continues, the system marks the results “Low Confidence.”
4. If the user imports more data, the flow returns to the Import Data step.